

Programmieren mit Python und Turtle

1 Zusammenfassung / Wiederholung

Wir haben in den letzten Wochen die Grundlagen der Programmierung mit Python kennengelernt. Dazu verwendeten wir die „Schildkröte“ in Python-Turtle welche wir mit bestimmten Befehlen bewegen können, um Linien auf dem Bildschirm zu zeichnen.

1.1 Grundlagen

Ein einfaches Turtle-Programm sieht zum Beispiel so aus:

```
1 from turtle import *
2
3 # Bewegt die Schildkroete um 100 Einheiten nach vorne
4 forward(100)
5
6 # Dreht die Schildkroete um 90 Grad nach rechts
7 right(90)
8
9 forward(100)
10 right(90)
11 forward(100)
12 right(90)
13 forward(100)
14 right(90)
15
16 # Sorgt dafuer, dass das Fenster am Ende nicht sofort verschwindet
17 mainloop()
```

Schnipsel 1: Simple Muster

Aufgabe 1: Kannst du sagen, was das Programm in Schnipsel 5 tut, ohne es auszuführen?

Aufgabe 2: Schreibe ein Programm, welches die Schildkröte den Buchstaben A zeichnen lässt.

Das erste Konzept der Programmierung, welches wir kennengelernt haben, ist die Verwendung von *Variablen*. Jede Variable hat einen Namen und einen *Wert*. Die beiden wichtigsten Vorteile von Variablen sind, dass unser Programmcode

- besser verständlich
- einfacher zu ändern

wird.

Beispiel:

```
1 from turtle import *
2
3 right(120)
4 forward(100)
5 right(120)
6 forward(100)
7 right(120)
```

```
8 forward(100)
9
10 mainloop()
```

Schnipsel 2: gleichseitiges Dreieck

Was tut dieses Programm? Wenn wir uns das Programm genau anschauen, sehen wir, dass 3 Drehungen mit anschließender Bewegung nach vorne geschehen, und somit ein gleichseitiges Dreieck gezeichnet wird. Um darauf zu kommen, müssen wir uns jede einzelne Zeile des Programms genau durchlesen.

1.2 Variablen

Vergleichen wir das Programm mit dieser Version:

```
1 from turtle import *
2
3 seitenlaenge = 100
4
5 right(120)
6 forward(seitenlaenge)
7 right(120)
8 forward(seitenlaenge)
9 right(120)
10 forward(seitenlaenge)
11
12 mainloop()
```

Schnipsel 3: gleichseitiges Dreieck

Jetzt ist viel klarer, was die 100 in unserem Programm bedeutet und auf einen Blick ersichtlich, dass 3 gleich lange Linien gezeichnet werden.

Allgemein werden Variablen in Python also wie folgt definiert:

```
1 [name] = [wert]
```

wobei wir `[name]` mit dem Namen der Variable ersetzen und `[wert]` mit dem Wert. Variablen können wir dann überall dort verwenden, wo wir den Wert direkt verwenden könnten.

Beispiele:

```
1 seitenlaenge = 350
2 forward(seitenlaenge)
3
4 winkel = 110
5 right(110)
6
7 name = "Mareike"
8 print(name)
```

Aufgabe 3: Schreibe ein Programm, welches ein Rechteck zeichnet. Die Höhe und die Breite des Rechtecks sollen dabei über jeweils eine eigene Variable änderbar sein.

1.3 Schleifen

Oft ist es so, dass wir für die Lösung eines Problems (also die Umsetzung eines Algorithmus) Schritte wiederholen müssen.

Beispiel: Ein Algorithmus, der beschreibt wie man „einmal um den Block fährt“, könnte wie folgt aussehen:

Algorithmus: BLOCK (1)

```
Fahre gerade aus bis zur nächsten Kreuzung
Biege nach rechts ab
Fahre gerade aus bis zur nächsten Kreuzung
Biege nach rechts ab
Fahre gerade aus bis zur nächsten Kreuzung
Biege nach rechts ab
Fahre gerade aus bis zur nächsten Kreuzung
Biege nach rechts ab
```

Natürlich könnten wir diesen Algorithmus wie folgt abkürzen:

Algorithmus: BLOCK (2)

```
[Tue die folgenden Dinge 4 mal]
| Fahre gerade aus bis zur nächsten Kreuzung
| Biege nach rechts ab
```



Dieser Algorithmus ist nicht korrekt (das heißt, er führt nicht immer zum richtigen Ergebnis), siehst du wieso? Kannst du ihn korrigieren?

Genau das gleiche können wir in Python mithilfe von Schleifen tun. Der Vorteil ist, dass wir uns nicht so oft wiederholen und wieder viel genauer sagen können was wir tun wollen. (Die Anweisung „*Mache 4 mal diese 2 Dinge: ...*“ ist wesentlich klarer als „*Mache diese 8 Dinge: ...*“)

Schauen wir noch einmal auf das Beispiel aus Schnipsel 5:

```
1 forward(100)
2 right(90)
3 forward(100)
4 right(90)
5 forward(100)
6 right(90)
7 forward(100)
8 right(90)
```

Schnipsel 4: Rechteck

Schauen wir genau hin, sehen wir dass die ersten beiden Zeilen vier mal wiederholt werden. In Python notieren wir das wie folgt:

```
1 for i in range(0, 4):
2     forward(100)
3     right(90)
```

Schnipsel 5: Simplex Muster

Das hat diese Bedeutung: `range(0, 4)` konstruiert ein Folge von Zahlen: 0, 1, 2, 3. Für jede dieser Zahlen wird der eingerückte Code „in“ der for-schleife je einmal ausgeführt.



„Einrücken“ bedeutet den Code nach rechts zu verschieben. Die Verschiebung muss in Python immer gleich sein (also zum Beispiel 0 Leerzeichen ganz links, 4 Leerzeichen auf der nächsten Ebene, und so weiter), sonst kann es zu Fehlern kommen. Am einfachsten geht es, wenn man mehrere Zeilen auswählt und dann „Tabulator“ (Doppelpfeil) drückt.



Bei `range(a, b)` ist die linke Zahl enthalten, die rechte aber nicht. Eine Schleife der Form `range(1, 5)` läuft also zum Beispiel 4 mal, nicht 5 mal.

Aufgabe 4: Schreibe ein Programm, welches ein reguläres 8-Eck zeichnet. Verwende dazu nicht mehr als 3 Zeilen Code (abgesehen von `from turtle import *` und `mainloop()`).



Tipp: Ein reguläres n -Eck hat eine Innenwinkelsumme von $(n - 2) \cdot 180$ Grad. Für ein 8-Eck ergibt sich damit 1080 Grad. Jeder einzelne Innenwinkel beträgt also $\frac{1080}{8} = 135$ Grad. Um wie viel Grad muss sich die Schildkroete nun drehen, um einen Innenwinkel von 135 Grad zu erzeugen?

2 Weiterführende Befehle

2.1 Gestaltung

Wir wollen uns ein paar Möglichkeiten ansehen, wie wird das Erscheinungsbild der Turtle-Zeichnung anpassen können.

Vorab: Haben wir ein langes Programm, sodass wir zu lange zuschauen müssen wie gezeichnet wird, können wir die Geschwindigkeit ändern. Das geht mit der Funktion `speed`. Wir suchen uns einfach die für unsere Situation passende Geschwindigkeit aus:

```
1 speed("fastest")
2 speed("fast")
3 speed("normal")
4 speed("slow")
5 speed("slowest")
```

Aufgabe 5: Probiere die Folgenden Befehle nacheinander (!) aus und schau, was diese bewirken.

```
1 shape("turtle")
2 pencolor("green")
3 width(5)
```

2.2 Weitere Bausteine

Wir haben bisher der Einfachheit halber fast nur die Befehle `forward` und `right` verwendet. Im Folgenden eine Liste mit weiteren Befehlen, die wir verwenden können, um kompliziertere Dinge zu zeichnen.

```
1 # Bringt die Schildkroete "nach Hause" (also zu den Koordinaten (0, 0))
2 home()
3
4 # Hebt den Stift an, die Kroete hrt also auf zu zeichnen wenn sie sich bewegt
5 penup()
6
7 # Senkt den Stift wieder, sodass wieder gezeichnet wird
8 pendown()
9
10 # Bewegt die Kroete vorwrts / rckwrts um die angegebene Distanz
11 forward(100)
12 backward(100)
13
14 # Dreht die Schildkroete im / gegen den Uhrzeigersinn um den angegebenen Winkel
15 right(80)
16 left(80)
17
18 # Zeichnet einen Kreis mit dem angegebenen Radius
19 circle(40)
20
21 # Malt einen "Punkt" mit der angegebenen Groesse und Farbe
22 dot(15, "blue")
23
24 # Teleportiert die Schildkroete (ohne zu zeichnen) an die angegebenen Koordinaten (x, y)
25 teleport(50, -30)
```

3 Aufgaben / Inspiration

Im Folgenden siehst du Bilder von Turtle-Zeichnungen. Versuche beliebige davon wie abgebildet oder leicht angepasst nachzuprogrammieren.

